

Java Learning Center

Hibernate

Ref. Guide

Doc Version - 1.0

Part 1

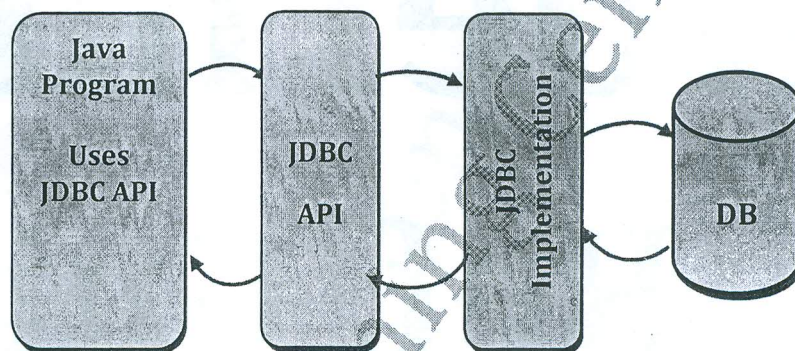
Kamlesh Kumar Singh

Author

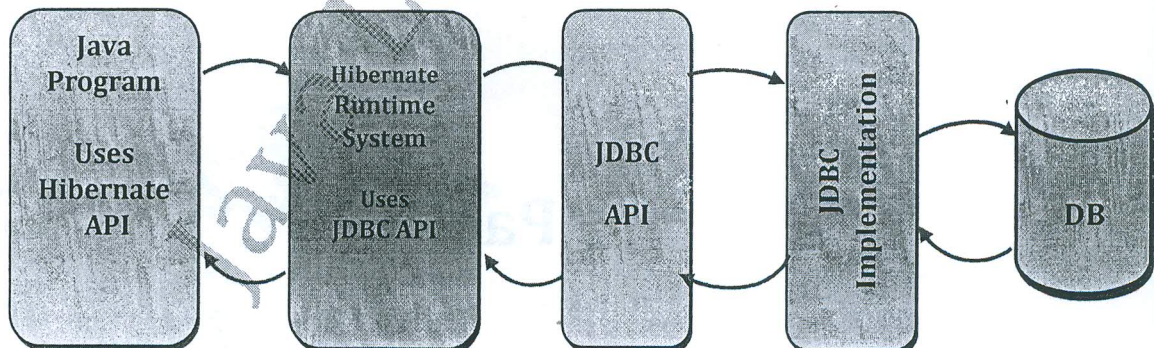
Srinivas Dande

Introduction

- ♦ Hibernate is a persistence framework which is used to implement persistence operations or Database operations.
- ♦ Hibernate framework was implemented by RedHat.
- ♦ Architect of Hibernate is Gavin King.
- ♦ Hibernate is ORM (Object Relational Mapping) tool and is best among all other persistence frameworks like iBatis, TopLink, JDO etc.
- ♦ Hibernate can be implemented either with XML approach (Hibernate Core) or Annotation approach (Hibernate Annotations).
- ♦ JDBC Specification is given by SUN and is implemented by various Database Vendors.



- ♦ Hibernate is implemented on the top of JDBC Technology i.e. you can use Hibernate API which internally uses JDBC to contact Database.



- ♦ When you write the JDBC code for implementing database operations, you need to perform the following:

```
try{
1.    Load the Driver Class
2.    Get the connection
3.    Prepare the SQL Statement (***)
4.    Create the JDBC Statement
5.    Submit the SQL Statement to DB.
6.    Process the Results (***)
}catch(Exception e){ }
finally{
7.    Clean the Resources
}
```

- ♦ In the above code, more statements (1, 2, 4, 5, 7) are common across multiple JDBC programs. This gives you the code duplication problem.
- ♦ In JDBC Java Developer is responsible to:
 - Generate Primary Keys.
 - Write SQL Statements.
 - Clean the resources.
 - Process the Results.
 - Implement Batch Updates.
 - etc

Hibernate Features

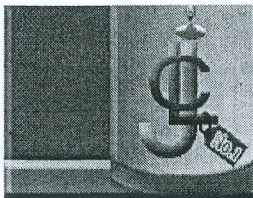
- 1) Hibernate system is responsible for taking the connections, creating statements and releasing the resources.

Ex:

```
Customer cust=new Customer("sri","sri@jlc",1234)
session.save(cust);
```

```
Customer cust=session.load(Customer.class,99)
```

- 2) Hibernate system is responsible for generating SQL queries which are well-tuned in terms of performance.
- 3) Hibernate system is responsible for generating the value required for Primary key columns.
- 4) Hibernate provides many built-In Primary key generation algorithms and also supports to implement Custom Primary key generation algorithms.



- 5) Hibernate supports various mapping styles:
 - 1) Simple Mapping
 - 2) Collection Mapping
 - 3) Inheritance Mapping
 - a. Table per sub class mapping
 - b. Table per class mapping
 - c. Table per concrete class mapping
 - 4) Association Mapping
 - a. One - to - One Mapping
 - b. One - to - Many Mapping
 - c. Many - to - Many Mapping
 - 5) Other mappings.
- 6) Hibernate supports two ways to manage connections
 - a. DriverManager connections
 - b. DataSource connections (*)
- 7) Hibernate supports two ways to manage Transactions
 - a. JDBC Transactions
 - b. JTA Transactions
- 8) Hibernate has in-built support for Batch Updates.
- 9) Hibernate provides various caching mechanism.
- 10) Hibernate provides various Query Languages:
 - a. HQL(*)
 - b. QBC(*)
 - c. QBE
 - d. Native SQL
 - e. Named SQL
- 11) Hibernate System uses many persistent best practices and forces the developer to use them for better performance.

Steps to develop First Hibernate Core Example in Eclipse

- 1) Create the Java Project with the name : Lab1
- 2) Add all hibernate core (3.1) jars to project build path.
- 3) Copy hibernate.cfg.xml to src folder.
Note : hibernate.cfg.xml is called Hibernate Configuration Document where you can specifying the Database details.
- 4) Consider the table called customers:

```
DROP DATABASE jlcindiadb;  
CREATE DATABASE jlcindiadb;  
USE jlcindiadb;
```



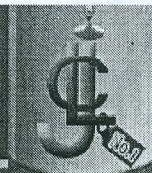
```
CREATE TABLE customers(
cid int primary key auto_increment, cname VARCHAR(15),
email VARCHAR(15), phone VARCHAR(15),
city VARCHAR(15), bal double );
```

- 5) Create a package called com.jlcindia.hibernate and write the following:
 - a. Hibernate Persistence Class i.e. Customer.java
 - b. Hibernate Mapping document i.e. Customer.hbm.xml to map your table with persistence class.
- 6) Write the following Hibernate Client code under package com.jlcindia.hibernate.
 - a. Lab1A.java
 - b. Lab1B.java

Lab1: Files required

1. Lab1A.java	2. Lab1B.java	3. Customer.java
4. Customer.hbm.xml	5. hibernate.cfg.xml	6. CHibernateUtil.java

1. Lab1A.java <pre>package com.jlcindia.hibernate; import org.hibernate.*; /* * @Author : Srinivas Dande * @company : Java Learning Center * @see : www.jlcindia.com */ public class Lab1A { public static void main(String[] args) { Transaction tx=null; try{ SessionFactory sf=CHibernateUtil.getSessionFactory(); Session session=sf.openSession(); tx=session.beginTransaction(); Customer cust=new Customer("Sri","Sri@jlc",1234,"Blore",25000.0); session.save(cust); tx.commit(); session.close(); System.out.println("Record Inserted"); }catch(Exception e){ e.printStackTrace(); if(tx!=null) tx.rollback(); } } }</pre>	2. Lab1B.java <pre>package com.jlcindia.hibernate; import org.hibernate.*; /* * @Author : Srinivas Dande * @company : Java Learning Center * @see : www.jlcindia.com */ public class Lab1B { public static void main(String[] args) { Transaction tx = null; try { SessionFactory sf = CHibernateUtil.getSessionFactory(); Session session = sf.openSession(); tx = session.beginTransaction(); Customer cust = (Customer) session.load(Customer.class, 1); System.out.println(cust.getCid() + "\t" + cust.getCName() + "\t" + cust.getEmail() + "\t" + cust.getPhone() + "\t" + cust.getCity() + "\t" + cust.getBal()); tx.commit(); session.close(); } catch (Exception e) { e.printStackTrace(); if (tx != null) tx.rollback(); } }}</pre>
---	--



3. Customer.java

```
package com.jlcindia.hibernate;
public class Customer {
    private int cid;           // P.K
    private String cname;
    private String email;
    private long phone;
    private String city;
    private double bal;

    public Customer() {}
    public Customer(String cname, String email, long phone,
        String city, double bal) {
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
        this.bal = bal;
    }

    // Setters and Getters
}
```

4. Customer.hbm.xml

```
<hibernate-mapping package="com.jlcindia.hibernate">
<class name="Customer" table="customers">
<id name="cid" column="cid" type="int">
<generator class="increment" />
</id>
<property name="cname" />
<property name="email" column="email" />
<property name="phone" column="phone" type="long"
/>
<property name="city" column="city" type="string" />
<property name="bal" column="bal" type="double" />
</class>
</hibernate-mapping>
```

5. hibernate.cfg.xml

```
<hibernate-configuration>
<session-factory>
<property name="hibernate.connection.driver_class">
    com.mysql.jdbc.Driver
</property>
<property name="hibernate.connection.url">
    jdbc:mysql://localhost:3306/jlcindiadb
</property>
<property name="hibernate.connection.username">
    root
</property>
<property name="hibernate.connection.password">
    srinivas
</property>
<property name="dialect">
    org.hibernate.dialect.MySQLDialect
</property>
<property name="show_sql">true</property>
<mapping
    resource="com/jlcindia/hibernate/Customer.hbm.xml" />
</session-factory>
</hibernate-configuration>
```

6. CHibernateUtil.java

```
package com.jlcindia.hibernate;
import org.hibernate.*;
import org.hibernate.cfg.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 * @see      : www.jlcindia.com
 */
public class CHibernateUtil {
    static SessionFactory factory;
    static {
        try {
            Configuration cfg = new Configuration();
            cfg = cfg.configure();
            factory = cfg.buildSessionFactory();
        } catch (Exception e) { e.printStackTrace(); }
    }
    public static SessionFactory getSessionFactory() {
        return factory;
    }
}
```

← Show SQL
these created
Hibernate
Internal

Steps to develop First Hibernate Annotation Example in Eclipse

- 1) Create the Java Project with the name : Lab2
- 2) Add all hibernate Annotation (3.2) jars to project build path.
- 3) Copy hibernate.cfg.xml to src folder.
Note : hibernate.cfg.xml is called Hibernate Configuration Document where you can specifying the Database details.
- 4) Consider the table called customers


```

DROP DATABASE jlcindiadb;
CREATE DATABASE jlcindiadb;
USE jlcindiadb;
CREATE TABLE customers(
    cid int primary key auto_increment, cname VARCHAR(15),
    email VARCHAR(15), phone VARCHAR(15),
    city VARCHAR(15), bal double );
      
```
- 5) Create a package called com.jlcindia.hibernate and write the following:
 - a. Hibernate Persistence Class i.e. Customer.java
- 6) Write the following Hibernate Client code under package com.jlcindia.hibernate.
 - a. Lab2A.java
 - b. Lab2B.java

Lab2: Files required

1. Lab2A.java	2. Lab2B.java	3. Customer.java
4. hibernate.cfg.xml	5. AHibernateUtil.java	

1. Lab2A.java <pre> package com.jlcindia.hibernate; import org.hibernate.*; /* * @Author : Srinivas Dande * @company : Java Learning Center * @see : www.jlcindia.com */ public class Lab2A { public static void main(String[] args) { Transaction tx=null; try{ SessionFactory sf = AHibernateUtil.getSessionFactory(); // Same as Lab1A.java }catch(Exception e){ e.printStackTrace(); if(tx!=null) tx.rollback(); } } } </pre>	2. Lab2B.java <pre> package com.jlcindia.hibernate; import org.hibernate.*; /* * @Author : Srinivas Dande * @company : Java Learning Center * @see : www.jlcindia.com */ public class Lab2B { public static void main(String[] args) { Transaction tx = null; try { SessionFactory sf = AHibernateUtil.getSessionFactory(); // Same as Lab1B.java } catch (Exception e) { e.printStackTrace(); if (tx != null) tx.rollback(); } } } </pre>
--	---

3. Customer.java

```
package com.jlcindia.hibernate;
import javax.persistence.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 * @see : www.jlcindia.com
 */

@Entity
@Table(name = "customers")
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "cid")
    private int cid; // P.K

    @Column(name = "cname")
    private String cname;

    @Column(name = "email")
    private String email;

    @Column(name = "phone")
    private long phone;

    @Column(name = "city")
    private String city;

    @Column(name = "bal")
    private double bal;

    public Customer() {}
    public Customer(String cname, String email, long phone,
String city, double bal) {
        this.cname = cname;
        this.email = email;
        this.phone = phone;
        this.city = city;
        this.bal = bal;
    }

    // Setters and Getters

}
```

4. hibernate.cfg.xml

```
<hibernate-configuration>
<session-factory>
<property name="hibernate.connection.driver_class">
    com.mysql.jdbc.Driver
</property>
<property name="hibernate.connection.url">
    jdbc:mysql://localhost:3306/jlcindiadb
</property>
<property name="hibernate.connection.username">
    root
</property>
<property name="hibernate.connection.password">
    srinivas
</property>
<property name="dialect">
    org.hibernate.dialect.MySQLDialect
</property>
<property name="show_sql">true</property>
<mapping class="com.jlcindia.hibernate.Customer" />
</session-factory>
</hibernate-configuration>
```

5. AHibernateUtil.java

```
package com.jlcindia.hibernate;
import org.hibernate.*;
import org.hibernate.cfg.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 * @see : www.jlcindia.com
 */

public class AHibernateUtil {
    static SessionFactory factory;
    static {
        AnnotationConfiguration cfg = new
        AnnotationConfiguration();
        cfg = (AnnotationConfiguration) cfg.configure();
        factory = cfg.buildSessionFactory();
    }

    public static SessionFactory getSessionFactory() {
        return factory;
    }
}
```




Questions

Q1) How many instances will be created for the persistence class with the following client code with Hibernate Core?

```
Configuration cfg=new Configuration();  
cfg=cfg.configure();
```

Ans: As per above code, 0 objects will be created.

Q2) How many instances will be created for the persistence class with the following client code with Hibernate Core?

```
Configuration cfg=new Configuration();  
cfg=cfg.configure();  
SessionFactory factory=cfg.buildSessionFactory();
```

Ans: 3 objects will be created using default constructor for the reflection process at the time of creating the SessionFactory object.

Q3) How many instances will be created for the persistence class with the following client code with Hibernate Core?

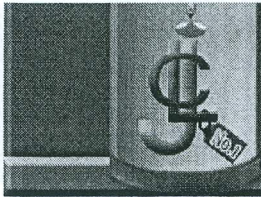
```
Configuration cfg=new Configuration();  
cfg=cfg.configure();  
SessionFactory factory=cfg.buildSessionFactory();  
Session session=factory.openSession();  
tx=session.beginTransaction();  
Customer c1=new Customer("sd","sd@jlc",1234,"Blore");  
session.save(c1);  
Customer c2=new Customer("ds","ds@jlc",4321,"Blore");  
session.save(c2);  
...
```

Ans: As per above code, 2 objects will be created with argumented constructor but at the time of creating the SessionFactory object, 3 objects will be created using default constructor for the reflection process.

Q4) How many instances will be created for the persistence class with the following client code with Hibernate Annotation?

```
AnnotationConfiguration cfg=new AnnotationConfiguration();  
cfg=(AnnotationConfiguration)cfg.configure();  
SessionFactory factory=cfg.buildSessionFactory();
```

Ans: 1 object will be created using default constructor for the reflection process at the time of creating the SessionFactory object.



Q5) How many instances will be created for the persistence class with the following client code with Hibernate Annotation?

```
AnnotationConfiguration cfg=new AnnotationConfiguration();
cfg=(AnnotationConfiguration)cfg.configure();
SessionFactory factory=cfg.buildSessionFactory();
Session session=factory.openSession();
tx=session.beginTransaction();
Customer c1=new Customer("sd","sd@jlc",1234,"Blore");
session.save(c1);
Customer c2=new Customer("ds","ds@jlc",4321,"Blore");
session.save(c2);
...
```

Ans: As per above code, 2 objects will be created with argumented constructor but at the time of creating the SessionFactory object, 1 object will be created using default constructor for the reflection process.

Q6) What will happen when I specify the field name wrongly with mapping document or Annotation?

Ans: You will get an exception `org.hibernate.PropertyNotFoundException: field not found: cname1`

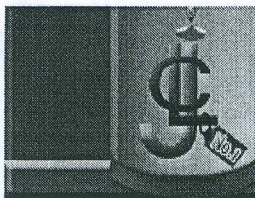
Q7) What will happen when I specify the column name wrongly with mapping document or Annotation ?

Ans : Unknown column 'cname1' in 'field list'

Hibernate Mappings

♦ Hibernate supports various mapping styles:

- 1) Simple Mapping
- 2) Collection Mapping
- 3) Inheritance Mapping
 - a. Table per sub class mapping
 - b. Table per class mapping
 - c. Table per concrete class mapping
- 4) Association Mapping
 - a. One - to - One Mapping
 - b. One - to - Many Mapping
 - c. Many- to - Many Mapping
- 5) Other mappings.



Simple Mapping

- When you map your Hibernate Persistence Class with simple data types like String, Primitives, Wrappers, Date etc with the corresponding table columns then it is called as Simple Mapping.

Ex:

```
public class Customer {  
    private int cid;  
    private String cname;  
    private String email;  
    ...  
}
```

With XML:

- For Primary Key Property, use <id> or <composite-id> tag.

Ex:

```
<id name="cid" column="cid" type="int">
```

} Primary key column can have special tag

- For other simple properties, use <property> tag.

```
<property name="cname" />
```

```
<property name="email" column="email" />
```

```
<property name="phone" column="phone" type="long" />
```

} Normal tag can use Property

With Annotation:

- You need to use following Annotations for Persistence class

@Entity

@Table(name = "customers")

class Customer { ... }

- You need to use following Annotations for primary key field

@Id

@Column(name="cid")

private int cid; //P.K

for P.K →

- You need to use following Annotations for other simple fields

@Column(name="cname")

private String cname;

Normal column →

Collection Mapping

When you map your Hibernate Persistence Class with collection data types like array, List, Set, Map then you have to use Collection mapping.

Example using Hibernate Core(Lab3):

A) Persistence Class

```

class Student{
    int sid; // P.K
    String sname;
    String dob;
    String qualification;
    String [] courses;
    List<String> emails;
    List<Integer> marks;
    Set<Long> phones;
    Map<String,Long> refs;
    ...
}
    
```

Note:- Collection type fields has to persisted in a Separate table always.

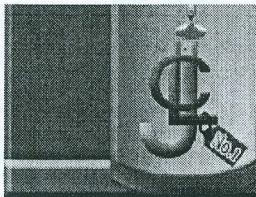
B) Tables

mystudents			
sid	sname	dob	qualification
101	Sri	25-12-1989	M.Sc

courses			emails		
sid	cname	idx	sid	emailid	idx
101	Java	0	101	aa@jlc.com	0
101	JDBC	1	101	bb@jlc.com	1
101	JSP	2	101	cc@jlc.com	2

marks		phones	
sid	marks	sid	phone
101	100	101	11111
101	99	101	2222
101	100	101	3333

refs		
sid	rname	rphone
101	aaa	1111
101	Bbb	2222
101	ccc	3333



Collection Mapping

When you map your Hibernate Persistence Class with collection data types like array, List, Set, Map then you have to use Collection mapping.

Example using Hibernate Core(Lab3):

A) Persistence Class

```
class Student{
    int sid; // P.K
    String sname;
    String dob;
    String qualification;
    String [] courses;
    List<String> emails;
    List<Integer> marks;
    Set<Long> phones;
    Map<String,Long> refs;
    ...
}
```

Note:- Collection type fields
has to persisted in a
separate table always.

B) Tables

mystudents			
sid	sname	dob	qualification
101	Sri	25-12-1989	M.Sc

courses		
sid	cname	idx
101	Java	0
101	JDBC	1
101	JSP	2

emails		
sid	emailid	idx
101	aa@jlc.com	0
101	bb@jlc.com	1
101	cc@jlc.com	2

marks	
sid	marks
101	100
101	99
101	100

phones	
sid	phone
101	11111
101	2222
101	3333

refs		
sid	rname	rphone
101	aaa	1111
101	Bbb	2222
101	ccc	3333



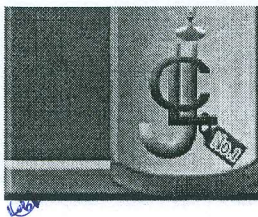
D) Client Code

```
String []crs={"Java","JDBC","JSP"};
List<String> ems=new ArrayList<String>();
ems.add("aa@jlc");
...
List<Integer> mks=new ArrayList<Integer>();
mks.add(100);
...
Set<Long> phs=new HashSet<Long>();
phs.add(new Long(1111));
...
Map<String,Long> refs=new HashMap<String,Long>();
refs.put("AAA",new Long(111));
....
Student stu=new Student("Sri","10-10-10","M.Sc",crs,ems,mks,phs,refs);

session.save(stu);
```

D) Mapping Document

```
<hibernate-mapping package="com.jlcindia.hibernate">
  <class name="Student" table="mystudents">
    <id name="sid" type="int" column="sid">
      <generator class="increment"/>
    </id>
    <property name="sname"/>
    <property name="dob"/>
    <property name="qualification" column="quali"/>
    <array name="courses" table="courses">
      <key column="sid"/>
      <index column="idx"/>
      <element column="cname" type="string"/>
    </array>
    <list name="emails" table="emails">
      <key column="sid"/>
      <index column="idx"/>
      <element column="email" type="string"/>
    </list>
    <bag name="marks" table="marks">
      <key column="sid"/>
      <element column="marks" type="int"/>
    </bag>
    <set name="phones" table="phones">
      <key column="sid"/>
      <element column="phone" type="long"/>
    </set>
    <map name="refs" table="refs">
      <key column="sid"/>
      <index column="rname" type="string"/>
      <element column="rphone" type="long"/>
    </map>
  </class> </hibernate-mapping>
```

Lab3: Files required

1. Lab3A.java	2. Lab3B.java	3. Student.java
4. Student.hbm.xml	5. hibernate.cfg.xml	6. CHibernateUtil.java

1. Lab3A.java

```
package com.jlcindia.hibernate;
import org.hibernate.*;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 * @see : www.jlcindia.com
 */
public class Lab3A {
    public static void main(String[] args) {
        try{
            SessionFactory sf=CHibernateUtil.getFactory();
            Session session=sf.openSession();
            Transaction tx=session.beginTransaction();

            String cous[]={"Java", "JDBC", "JSP"};
            List<String> ems=new ArrayList<String>();
            ems.add("aa@jlc");
            ems.add("bb@jlc");
            ems.add("cc@jlc");

            List<Integer> maks=new ArrayList<Integer>();
            maks.add(new Integer(100));
            maks.add(new Integer(99));
            maks.add(new Integer(100));

            Set<Long> phs=new HashSet<Long>();
            phs.add(new Long(1111));
            phs.add(new Long(2222));
            phs.add(new Long(3333));

            Map<String,Long> refs=new HashMap<String,Long>();
            refs.put("aaa",new Long(1111));
            refs.put("bbb",new Long(2222));
            refs.put("ccc",new Long(3333));
            Student stu=new Student("sri", "10-10-10", "M.Sc",cous,ems,maks,phs,refs);
            session.save(stu);
            tx.commit();
            session.close();
        }catch(Exception e){
            e.printStackTrace();
        }
    }
}
```

2. Lab3B.java

```
package com.jlcindia.hibernate;
import org.hibernate.*;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 * @see : www.jlcindia.com
 */
public class Lab3B{
    public static void main(String[] args) {
        try{
            SessionFactory sf=CHibernateUtil.getFactory();
            Session session=sf.openSession();
            Transaction tx=session.beginTransaction();

            Student stu=(Student)session.load(Student.class, 1);
            System.out.println(stu.getSid()+"\t"+stu.getSname()+"\t"+stu.getDob());
            for(String cn:stu.getCourses()){
                System.out.println(cn);
            }
            System.out.println(stu.getEmails());
            System.out.println(stu.getMarks());
            System.out.println(stu.getPhones());
            System.out.println(stu.getRefs());

            tx.commit();
            session.close();
        }catch(Exception e){
            e.printStackTrace();
        }
    }
}
```


Persistence class

3. Student.java

```
package com.jlcindia.hibernate;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 * @see : www.jlcindia.com
 */
public class Student {
    private int sid; // P.K
    private String sname;
    private String dob;
    private String qualification;
    private String [] courses;
    private List<String> emails;
    private List<Integer> marks;
    private Set<Long> phones;
    private Map<String,Long> refs;
    public Student(){}
    public Student(String sname, String dob, String
    qualification,
    String[] courses, List<String> emails, List<Integer>
    marks,
    Set<Long> phones, Map<String, Long> refs) {
        super();
        this.sname = sname;
        this.dob = dob;
        this.qualification = qualification;
        this.courses = courses;
        this.emails = emails;
        this.marks = marks;
        this.phones = phones;
        this.refs = refs;
    }
    // setters and getters
}
```

*if array type
variable
PK
Index of
column*

5. hibernate.cfg.xml

```
<hibernate-configuration>
<session-factory>
....
<property name="hbm2ddl.auto">update</property>
<mapping
resource="com/jlcindia/hibernate/Student.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

4. Student.hbm.xml

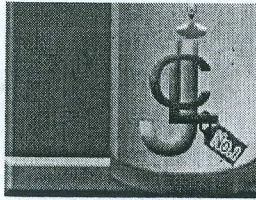
```
<hibernate-mapping package="com.jlcindia.hibernate">
<class name="Student" table="students">
<id name="sid" column="sid" type="int">
<generator class="increment"/>
</id>
<property name="sname"/>
<property name="dob"/>
<property name="qualification" column="quali"/>
<array name="courses" table="courses">
<key column="sid"/>
<index column="idx"/>
<element column="cname" type="string"/>
</array>
<list name="emails" table="emails">
<key column="sid"/>
<index column="idx"/>
<element column="emailId" type="string"/>
</list>
<bag name="marks" table="marks">
<key column="sid"/>
<element column="marks" type="int"/>
</bag>
<set name="phones" table="phones">
<key column="sid"/>
<element column="phoneNo" type="long"/>
</set>
<map name="refs" table="refs">
<key column="sid"/>
<index column="rname" type="string"/>
<element column="rphone" type="long"/>
</map>
</class>
</hibernate-mapping>
```

*← No index column
List type variable doesn't have index
column. then used bag*

*if Table is available is database then it will update
data in the table other wise it can create
the table and update the value.*

index

PK	Chai	idx
0	A	0
1	B	1
2	C	2



Example using Hibernate Annotation(Lab4):

A) Persistence Class

```
@Entity
@Table(name="mystudents")
public class Student {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    @Column(name="sid")
    private int sid; // P.K

    @Column(name="sname")
    private String sname;

    @Column(name="dob")
    private String dob;

    @Column(name="quali")
    private String qualification;

    @CollectionOfElements
    @JoinTable(name="courses",joinColumns=@JoinColumn(name="sid"))
    @IndexColumn(name="idx")
    @Column(name="cname")
    private String [] courses;

    @CollectionOfElements
    @JoinTable(name="emails",joinColumns=@JoinColumn(name="sid"))
    @IndexColumn(name="idx")
    @Column(name="emailId")
    private List<String> emails;

    @CollectionOfElements
    @JoinTable(name="marks",joinColumns=@JoinColumn(name="sid"))
    @Column(name="marks")
    private List<Integer> marks;

    @CollectionOfElements
    @JoinTable(name="phones",joinColumns=@JoinColumn(name="sid"))
    @Column(name="phoneNo")
    private Set<Long> phones;

    @CollectionOfElements
    @JoinTable(name="refs",joinColumns=@JoinColumn(name="sid"))
    @Column(name="rphone")
    private Map<String,Long> refs;
    ...
}
```

JPA can't support collection element
so that you are using
prog. Hibernate

1st
2nd
3rd

only name=phone
are required
name are
not required
to write